Pair Programming AI Assistant — Refactoring: SRP & Modularity

KK

REFACTOR PROMPT

Review `addTask` for Single Responsibility Principle (SRP) violations:

- Identify all SRP violations (task management vs state checking, scheduling, and logging).
- Refactor by extracting logging/scheduling out into a separate class/component so `addTask` is strictly focused on task insertion.
- Explain how this refactoring improves testability and maintainability.

AI

SRP Analysis & Refactoring Solution

1. Identified SRP Violations

The legacy `addTask` method attempted to perform 4 distinct responsibilities:

- **Task validation & storage:** Validating `taskFn` and pushing into `this.tasks`.
- **Queue State Evaluation:** Inspecting queue length to trigger processing (`this.tasks.length === 1`).
- **Execution Scheduling:** Spawning background execution via `this._startProcessing()`.
- **Notification & Alerting:** Console output and high-priority alerts via `notify()`.

2. Refactored Implementation (Extracted Notifier & Clean Queue)

```
class QueueNotifier {
 notifyQueueStarted(queueName) {
   console.log(`Starting queue ${queueName}.`);
 }
 notifyHighPriorityTask(queueName, priority) {
   if (priority > 9) {
     console.warn(`High priority task added to ${queueName}.`);
   }
 }
 logError(message) {
   console.error(message);
 }
```